Ministerul Educației și Cercetării Al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 1: Study and Empirical Analysis of Algorithms for Determining Fibonacci N-th Term

Elaborated:
st. gr. FAF-242 Canțer Daniel

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2026

Table of Contents

ALGORITHM ANALYSIS **3**
Objective 3
Tasks: 3
Theoretical Notes: 3
Introduction: 4
Comparison Metric: 4
Input Format: 4
IMPLEMENTATION **5**
Recursive Method: 5
Dynamic Programming Method: 7
Matrix Power Method: 8
Binet Formula Method: 11
Fast Doubling Method: 13
CONCLUSION **16**

ALGORITHM ANALYSIS

Objective

Study and analyze different algorithms for determining Fibonacci n-th term.

Tasks:

1. Implement at least 3 algorithms for determining Fibonacci n-th term;
2. Decide properties of input format that will be used for algorithm analysis;
3. Decide the comparison metric for the algorithms;
4. Analyze empirically the algorithms;
5. Present the results of the obtained data;
6. Deduce conclusions of the laboratory.

Theoretical Notes:

An alternative to mathematical analysis of complexity is empirical analysis.

This may be useful for: obtaining preliminary information on the complexity class of an algorithm; comparing the efficiency of two (or more) algorithms for solving the same problems; comparing the efficiency of several implementations of the same algorithm; obtaining information on the efficiency of implementing an algorithm on a particular computer.

In the empirical analysis of an algorithm, the following steps are usually followed:

1. The purpose of the analysis is established.
2. Choose the efficiency metric to be used (number of executions of an operation (s) or time execution of all or part of the algorithm.
3. The properties of the input data in relation to which the analysis is performed are established (data size or specific properties).
4. The algorithm is implemented in a programming language.
5. Generating multiple sets of input data.
6. Run the program for each input data set.
7. The obtained data are analyzed.

The choice of the efficiency measure depends on the purpose of the analysis. If, for example, the aim is to obtain information on the complexity class or even checking the accuracy of a theoretical estimate then it is appropriate to use the number of operations performed. But if the goal is to assess the behavior of the implementation of an algorithm then execution time is appropriate.

After the execution of the program with the test data, the results are recorded and, for the purpose of the analysis, either synthetic quantities (mean, standard deviation, etc.) are calculated or a graph with appropriate pairs of points (i.e. problem size, efficiency measure) is plotted.

Introduction:

The Fibonacci sequence is the series of numbers where each number is the sum of the two preceding numbers. For example: $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, \ldots$ Mathematically we can describe this as: $x_n = x_{n-1} + x_{n-2}$.

Many sources claim this sequence was first discovered or "invented" by Leonardo Fibonacci. The Italian mathematician, who was born around A.D. 1170, was initially known as Leonardo of Pisa. In the 19th century, historians came up with the nickname Fibonacci (roughly meaning "son of the Bonacci clan") to distinguish the mathematician from another famous Leonardo of Pisa. There are others who say he did not. Keith Devlin, the author of Finding Fibonacci: The Quest to Rediscover the Forgotten Mathematical Genius Who Changed the World, says there are ancient Sanskrit texts that use the Hindu-Arabic numeral system - predating Leonardo of Pisa by centuries. But, in 1202 Leonardo of Pisa published a mathematical text, Liber Abaci. It was a "cookbook" written for tradespeople on how to do calculations. The text laid out the Hindu-Arabic arithmetic useful for tracking profits, losses, remaining loan balances, etc, introducing the Fibonacci sequence to the Western world.

Traditionally, the sequence was determined just by adding two predecessors to obtain a new number, however, with the evolution of computer science and algorithmics, several distinct methods for determination have been uncovered. The methods can be grouped in 5 categories, Recursive Methods, Dynamic Programming Methods, Matrix Power Methods, Binet Formula Methods and Fast Doubling Methods. All those can be implemented naively or with a certain degree of optimization, that boosts their performance during analysis.

As mentioned previously, the performance of an algorithm can be analyzed mathematically (derived through mathematical reasoning) or empirically (based on experimental observations). Within this laboratory, we will be analyzing the 5 naïve algorithms empirically.

Comparison Metric:

The comparison metric for this laboratory work will be considered the time of execution of each algorithm ($O(n)$)

Input Format:

As input, each algorithm will receive two series of numbers that will contain the order of the Fibonacci terms being looked up. The first series will have a more limited scope, $(5, 7, 10, 12, 15, 17, 20, 22, 25, 27, 30, 32, 35, 37, 40, 42, 45)$, to accommodate the recursive method, while the second series will have a bigger scope to be able to compare the other algorithms between themselves $(501, 631, 794, 1000, 1259, 1585, 1995, 2512, 3162, 3981, 5012, 6310, 7943, 10000, 12589, 15849)$.

IMPLEMENTATION

All five algorithms will be implemented in their naïve form in C with the help of GMP library (GNU Multiple Precision Arithmetic Library) to being able to handle large numbers efficiently and analyzed empirically based on the time required for their completion. To get better results from execution times, the CPU times will be measured and furthermore the algorithms are run a few times and averaged. While the general trend of the results may be similar to other experimental observations, the particular efficiency in rapport with input will vary depending on memory and the speed of the device used.

The error margin determined will constitute 2.5 seconds as per experimental measurement.

Recursive Method:

The recursive method, also considered the most inefficient method, follows a straightforward approach of computing the n-th term by computing it's predecessors first, and then adding them. However, the method does it by calling upon itself a number of times and repeating the same operation, for the same term, at least twice, occupying additional memory and, in theory, doubling it's execution time.

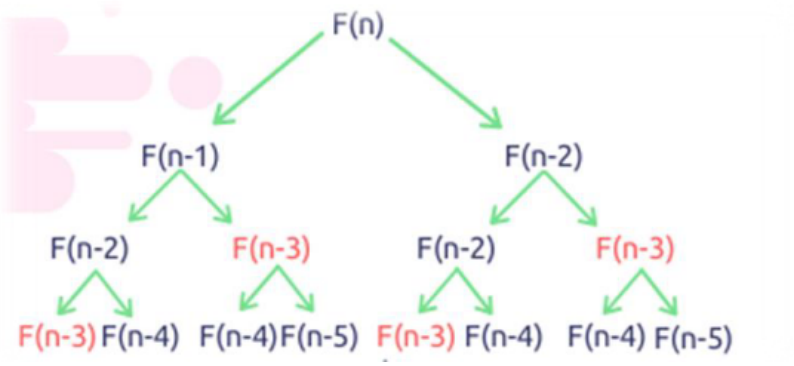

Figure 1 Fibonacci Recursion

Algorithm Description:

The naïve recursive Fibonacci method follows the algorithm as shown in the next pseudocode:

```
Fibonacci(n):
    if n <= 1:
        return n
    otherwise:
        return Fibonacci(n-1) + Fibonacci(n-2)
```

Implementation:

```
void recursive_fibo(mpz_t result, uint64_t n) {
    if (n <= 1) {
        mpz_set_ui(result, n);
        // return the number if it's less than or equal to 1
        return;
    }
    // Initialize variables for the recursive calls
    mpz_t a, b;
    mpz_init(a);
    mpz_init(b);
    recursive_fibo(a, n - 1); //recursive call for n-1
    recursive_fibo(b, n - 2); //recursive call for n-2
    // Add the results of the recursive calls
    mpz_add(result, a, b);
    // Clear the temporary variables
    mpz_clear(a);
    mpz_clear(b);
}
```

Results:

After running the function for each n Fibonacci term proposed in the list from the first Input Format and saving the time for each n, we obtained the following results:

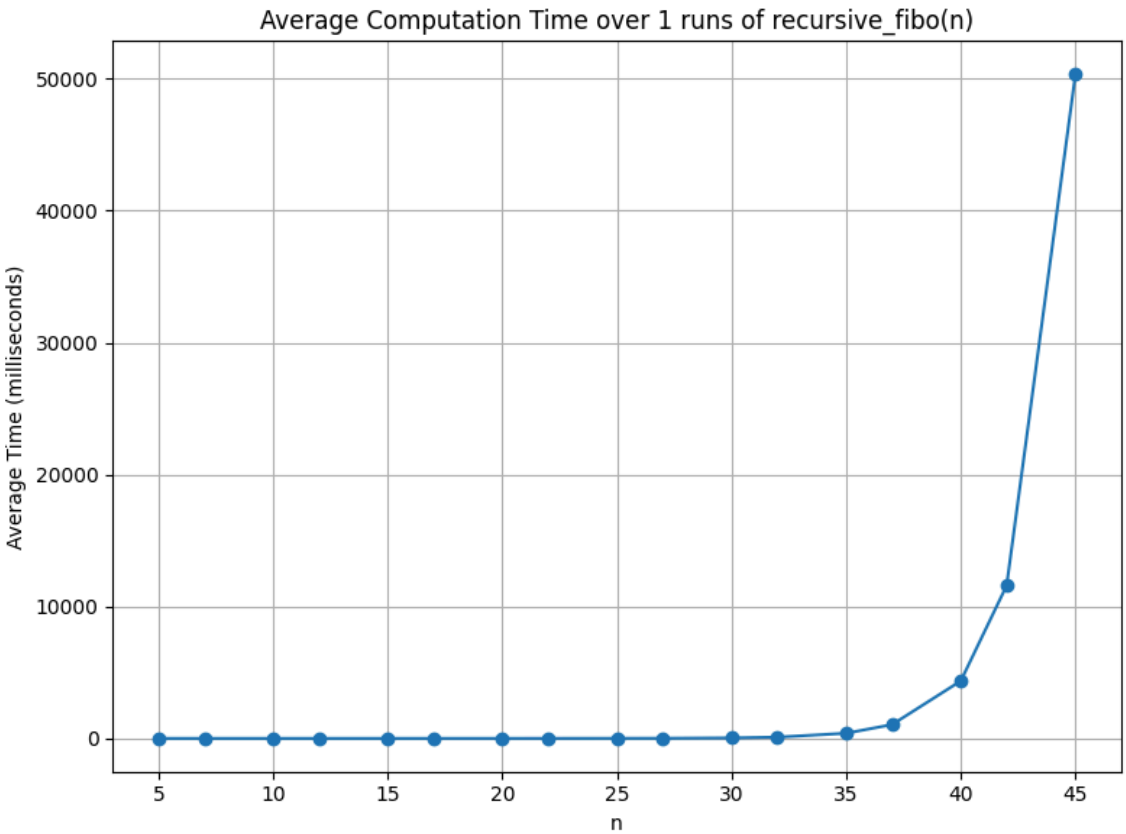

Figure 2 Results for first set of inputs

In the graph Figure 2 we can observe the growth of the time needed for the operations, we may easily see the spike in time complexity that happens after the 42nd term, leading us to deduce that the Time Complexity is exponential. $T(2^n)$.

Dynamic Programming Method:

The Dynamic Programming method, similar to the recursive method, takes the straightforward approach of calculating the n-th term. However, instead of calling the function upon itself, from top down it operates based on an array data structure that holds the previously computed terms, eliminating the need to recompute them.

Algorithm Description:

The naïve DP algorithm for Fibonacci n-th term follows the pseudocode:

```
Fibonacci(n):
    Array A;
    A[0]<-0;
    A[1]<-1;
    for i <- 2 to n - 1 do
        A[i]<-A[i-1]+A[i-2];
    return A[n-1]
```

Implementation:

```
void dp_fibo(mpz_t result, uint64_t n) {
    if (n <= 1) { // If n is less than or equal to 1 return
        mpz_set_ui(result, n);
        return;
    }
    mpz_t *vec = malloc(sizeof(mpz_t) * (n + 1)); //Initializ the array
    for (int i = 0; i < n + 1; i++) {
        mpz_init(vec[i]);
    }
    mpz_set_ui(vec[0], 0); //Set the first term
    mpz_set_ui(vec[1], 1); //Set the second term
    for (int i = 2; i < n + 1; ++i) {
        mpz_add(vec[i], vec[i - 1], vec[i - 2]); //Calculate
    }
    mpz_set(result, vec[n]); // Return the result
    for (int i = 0; i < n + 1; i++) {
        mpz_clear(vec[i]);
    }
    free(vec);
}
```

After the execution of the function for each n Fibonacci term mentioned in the second set of Input Format we obtain the following results:

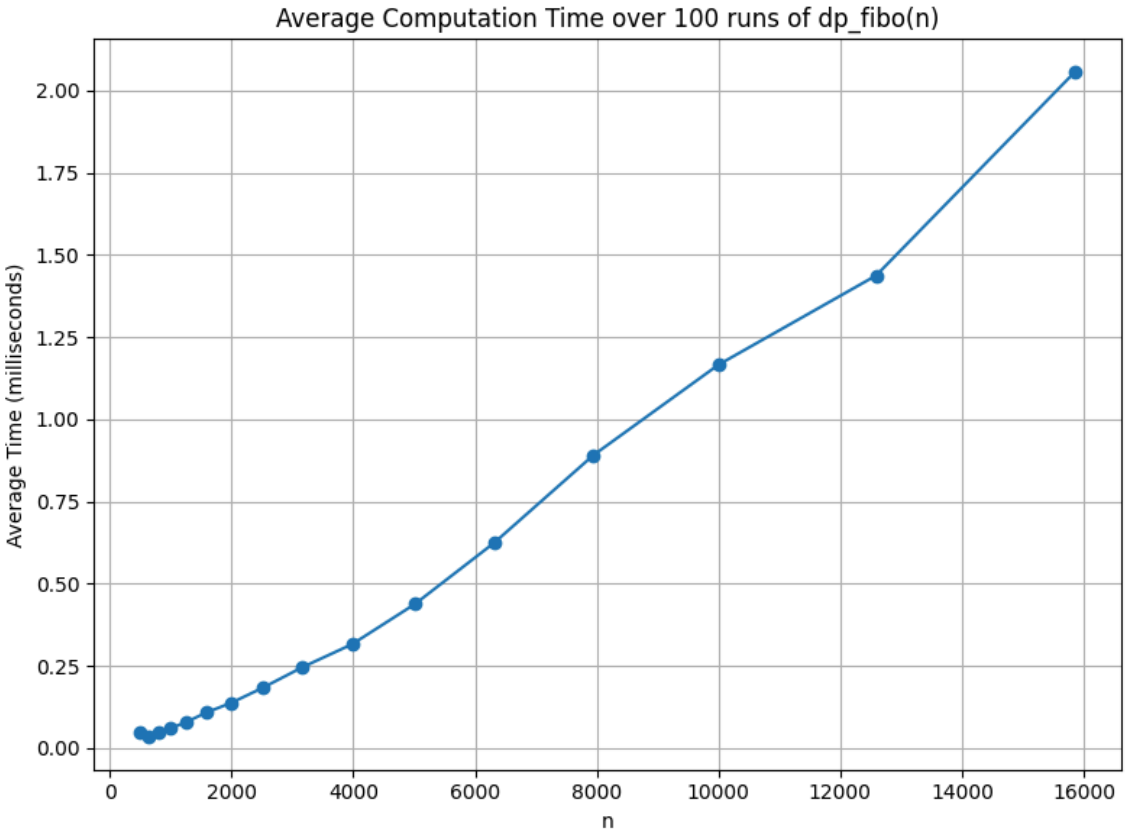

Figure 3 Fibonacci DP Results

From the resulting graph we can clearly observe that the time complexity of the dynamic programming is linear $O(n)$.

Matrix Power Method:

The Matrix Power method of determining the n-th Fibonacci number is based on, as expected, the multiple multiplication of a naïve Matrix $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ with itself.

Algorithm Description: It is known that

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ a+b \end{pmatrix}$$

This property of Matrix multiplication can be used to represent

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_1 \\ F_2 \end{pmatrix}$$

And similarly:

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^2 \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_2 \\ F_3 \end{pmatrix}$$

Which turns into the general:

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}$$

This set of operation can be described in pseudocode as follows:

```
Fibonacci(n):
    F<- []
    vec <- [[0], [1]]
    Matrix <- [[0, 1],[1, 1]]
    F <-power(Matrix, n)
    F <- F * vec
    Return F[0][0]
```

Implementation:

The implementation of the driving function in C is as follows:

```c
void matrix_fibo(mpz_t result, uint64_t n) {
    if (n <= 1) { // return early
        mpz_set_ui(result, n);
        return;
    }
    mpz_t f[2][2]; // Setup the matrix
    mpz_init_set_ui(f[0][0], 1);
    mpz_init_set_ui(f[0][1], 1);
    mpz_init_set_ui(f[1][0], 1);
    mpz_init_set_ui(f[1][1], 0);
    power(f, n - 1); // Compute the power of the matrix
    mpz_set(result, f[0][0]); //Return the first element
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            mpz_clear(f[i][j]);
}
void power(mpz_t f[2][2], uint64_t n) {
    if (n <= 1) return;
    mpz_t m[2][2];// Initialize the matrix with initial same values
    mpz_init_set_ui(m[0][0], 1);
    mpz_init_set_ui(m[0][1], 1);
    mpz_init_set_ui(m[1][0], 1);
    mpz_init_set_ui(m[1][1], 0);
    for (uint64_t i = 1; i < n; i++) {
        multiply(f, m); // Multiply the matrix with itself n times
    }
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            mpz_clear(m[i][j]);
}
```

Where the power function handles the part of raising the Matrix to the power n, while the multiplying function handles the matrix multiplication with itself.

```
void multiply(mpz_t f[2][2], mpz_t m[2][2]) {
    mpz_t res[2][2]; // Initialize result matrix
    for (int i = 0; i < 2; ++i)
        for (int j = 0; j < 2; ++j)
            mpz_init(res[i][j]);
    mpz_t temp;
    mpz_init(temp);
    for (int i = 0; i < 2; ++i) { // Matrix multiplication
        for (int j = 0; j < 2; ++j) {
            mpz_set_ui(res[i][j], 0);
            for (int k = 0; k < 2; ++k) {
                mpz_mul(temp, f[i][k], m[k][j]);
                mpz_add(res[i][j], res[i][j], temp);
            }
        }
    }
    mpz_clear(temp);
    for (int i = 0; i < 2; ++i) //Set the result matrix
        for (int j = 0; j < 2; ++j)
            mpz_set(f[i][j], res[i][j]);
    for (int i = 0; i < 2; ++i) //Clear the result matrix
        for (int j = 0; j < 2; ++j)
            mpz_clear(res[i][j]);
}
```

Results:

After the execution of the function for each n Fibonacci term mentioned in the second set of Input Format we obtain the following results:

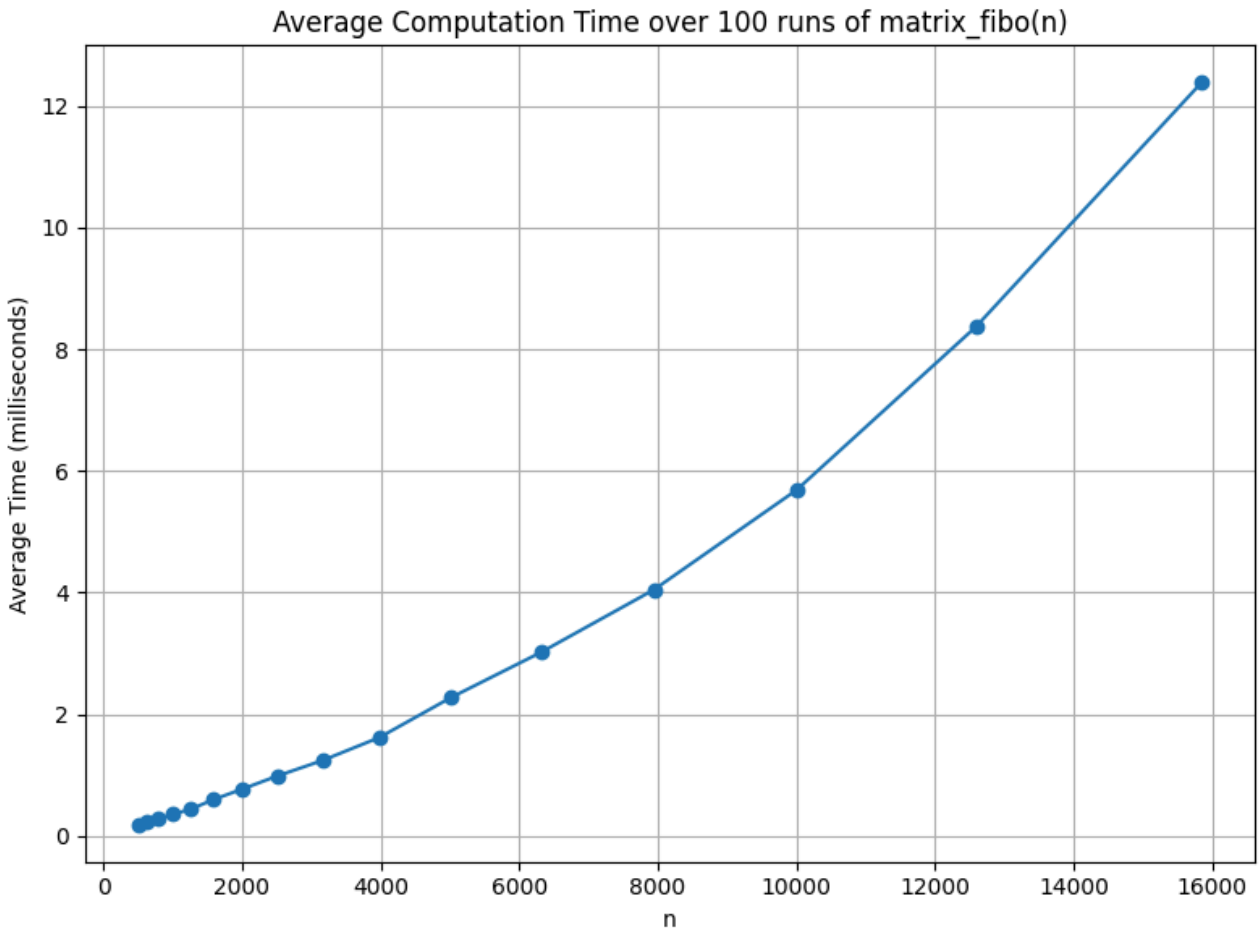

Figure 4 Matrix Method Fibonacci Results

With the matrix method we also can see that the time complexity is linear $O(n)$. This is because we implement the matrix multiplication in a naive way and if we would consider the optimized version of matrix multiplication, the time complexity would be reduced to $O(\log(n))$.

Binet Formula Method:

The Binet Formula Method is another unconventional way of calculating the n-th term of the Fibonacci series, as it operates using the Golden Ratio formula, or φ. However, due to its nature of requiring the usage of decimal numbers, at some point, the rounding error of uint64 that accumulates, begins affecting the results significantly. The observation of error starting with around 70th number making it unusable in practice, despite its speed.

Algorithm Description:

The set of operation for the Binet Formula Method can be described in pseudocode as follows:

```
Fibonacci(n):
    phi <- (1 + sqrt(5))
    phi1 <-(1 - sqrt(5))
    return pow(phi, n)- pow(phi1, n)/(pow(2, n)*sqrt(5))
```

Implementation:

The implementation of the function in C is as follows, with some alterations that would increase the number of terms that could be obtain through it:

```
void binet_fibo(mpz_t result, uint64_t n) {
    if (n <= 1) { // Return early
        mpz_set_ui(result, n);
        return;
    }
    mpf_t phi_mpf, phi1_mpf, sqrt5_mpf, result_mpf, temp_mpf, mpf_2;
    binet_fibo_init(phi_mpf, phi1_mpf, sqrt5_mpf, result_mpf, temp_mpf, mpf_2); //
Initialize variables
    mpf_set_default_prec(256);
    //Use the Binet formula to calculate the nth Fibonacci number
    mpf_pow_ui(result_mpf, phi_mpf, n);
    mpf_pow_ui(temp_mpf, phi1_mpf, n);
    mpf_sub(result_mpf, result_mpf, temp_mpf);
    mpf_pow_ui(temp_mpf, mpf_2, n);
    mpf_mul(temp_mpf, temp_mpf, sqrt5_mpf);
    mpf_div(result_mpf, result_mpf, temp_mpf);
    mpz_set_f(result, result_mpf);//Return the result
    mpf_clear(result_mpf);//Cleanup
    mpf_clear(phi_mpf);
    mpf_clear(phi1_mpf);
    mpf_clear(sqrt5_mpf);
}
```

Results:

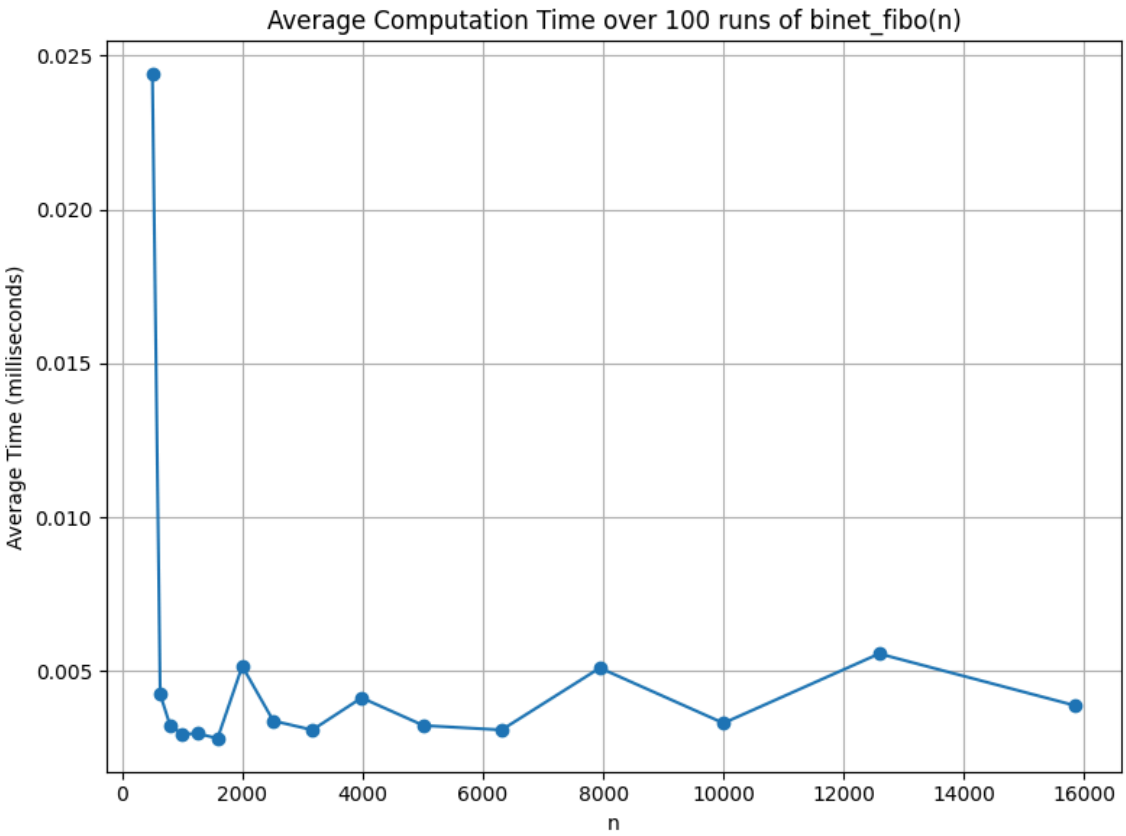

Figure 5 Fibonacci Binet Formula Method results

Although the most performant with its time, the Binet Formula Function is not accurate enough to be considered within the analysed limits and is recommended to be used for Fibonacci terms up to 80. At least in its naïve form in C, this could have been improved by calculating the constants with a higher precision. The formula is in constant time as we can also see from the Figure 5 if we would ignore the inital CPU optimization spike.

Fast Doubling Method:

The Fast Doubling Method is an improvement over the Matrix Method. And the following identities can be extracted from the Matrix Formula Method:

$$F(2k) = F(k)[2F(k+1) - F(k)]$$

$$F(2k+1) = F(k+1)^2 + F(k)^2$$

Algorithm Description:

The set of operation for the Binet Formula Method can be described in pseudocode as follows:

```
(Note that this function returns a touple with Fib_n and Fib_(n+1))
Fibonacci(n):
    if n == 0:
        vec<-(0,1)
        return
    else:
        a, b <- Fibonacci(⌊n/2⌋)
        c <- a * (b * 2 -a)
        d <- a * a + b * b
        if n % 2 == 0:
            return (c, d)
        else:
            return (d, c + d)
```

Implementation:

The implementation of the function in C is as follows, with a helper function to help return one value:

```
void doubling_fibo(mpz_t result, uint64_t n) {
    if (n <= 1) { //Return early
        mpz_set_ui(result, n);
        return;
    }
    touple t = _doubling_fibo(n); //Call the function
    mpz_set(result, t.t[0]); //Return the first element Fib_n
}
touple _doubling_fibo(uint64_t n) {
    touple t;
    mpz_init_set_ui(t.t[0], n);
    mpz_init_set_ui(t.t[1], n);
    if (n <= 1) { //return early
        return t;
    }
    touple p = _doubling_fibo(n >> 1);// get the n/2 floored using bit shift
    mpz_t temp, c, d;
    mpz_init(temp);
    //c F(2k)=F(k)[2F(k+1)-F(k)]
    mpz_mul_ui(temp, p.t[1], 2);
    mpz_sub(temp, temp, p.t[0]);
    mpz_mul(temp, p.t[0], temp);
    mpz_init_set(c, temp);
    //d F(2k+1)=F(k+1)^2+F(k)^2
    mpz_pow_ui(temp, p.t[0], 2);
    mpz_init_set(d, temp);
    mpz_pow_ui(temp, p.t[1], 2);
    mpz_add(d, d, temp);
    if (n % 2) { //odd case
```

```
        mpz_set(t.t[0], d);
        mpz_add(t.t[1], c, d);
        return t;
    } else { //even case
        mpz_set(t.t[0], c);
        mpz_set(t.t[1], d);
        return t;
    }
}
```

Results:

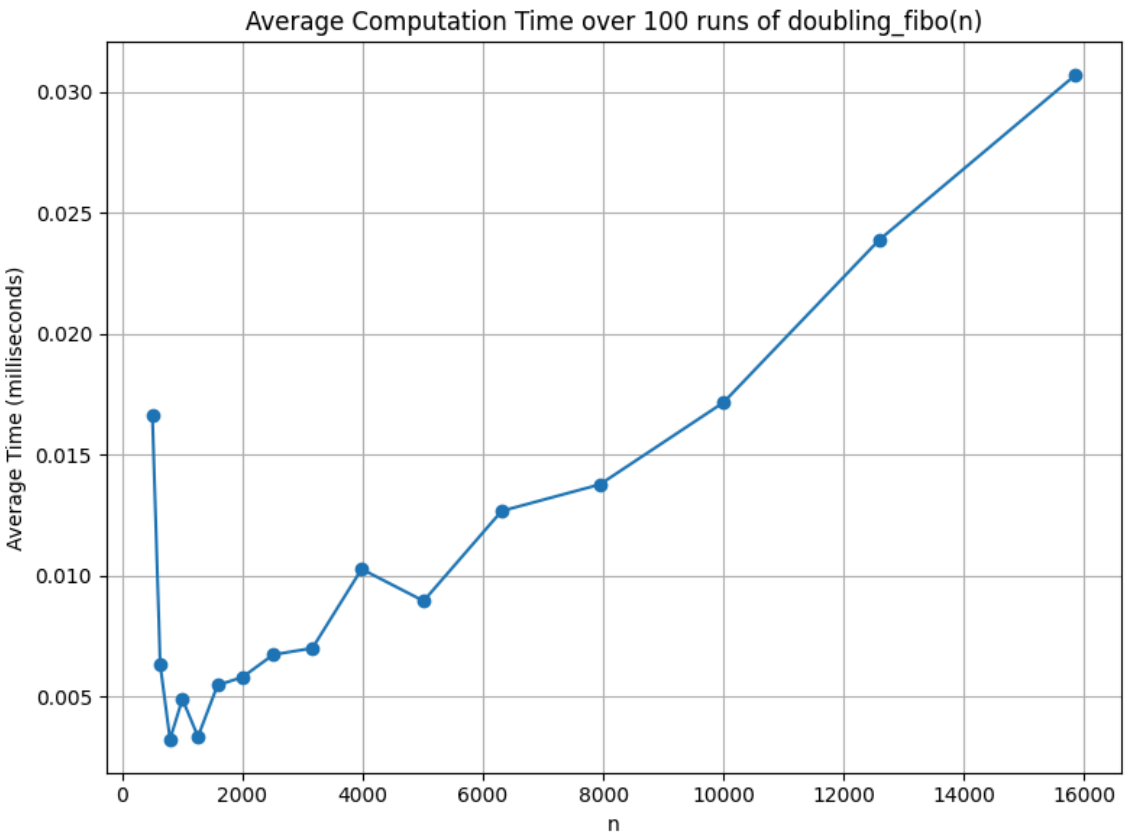

Figure 6 Fibonacci Fast Doubling Formula Method results

Although not that fast as Binet Formula Method, the Fast Doubling Formula is one of the best formulas with its $O(\log(n))$ time complexity.

CONCLUSION

Through Empirical Analysis, within this paper, five classes of methods have been tested in their efficiency at both their providing of accurate results, as well as at the time complexity required for their execution, to delimit the scopes within which each could be used, as well as possible improvements that could be further done to make them more feasible.

The Recursive method, being the easiest to write, but also the most difficult to execute with an exponential time complexity, can be used for smaller order numbers, such as numbers of order up to 30 with no additional strain on the computing machine and no need for testing of patience.

The Binet method, the easiest to execute with an almost constant time complexity, could be used when computing numbers of order up to 80, after the recursive method becomes unfeasible. However, its results are recommended to be verified depending on the language used, as there could rounding errors due to its formula that uses the Golden Ratio.

The Dynamic Programming and Matrix Multiplication Methods can be used to compute Fibonacci numbers further then the ones specified above, both of them presenting exact results and showing a linear complexity in their naivety that could be, with additional tricks and optimisations, reduced to logarithmic.

The Fast Doubling Formula method, being the easiest to execute with a logarithmic time complexity, could be used when computing very large numbers of the series.

Github Repository: https://github.com/The1Dani/AA-LABS